

Relatório Técnico — Simulador de Sobrevivência

1. Objetivo

O trabalho consiste em um sistema de simulação de sobrevivência, desenvolvido em Python, que permite cadastrar sobreviventes, coletar recursos, construir itens, controlar um inventário e lidar com eventos aleatórios. O programa foi construído inteiramente com recursos da biblioteca padrão da linguagem, sem uso de bibliotecas externas, podendo ser executado diretamente pelo interpretador do Python.

2. Passos seguidos para solução do problema

- 1. Levantamento dos requisitos:** primeiro foram identificadas as cinco funcionalidades obrigatórias (cadastro de sobreviventes, coleta de recursos, construção de itens, controle de inventário e eventos aleatórios) e os recursos técnicos exigidos (listas aninhadas, matrizes e funções).
- 2. Modelagem das estruturas de dados:** antes de escrever qualquer função, foi definido como cada informação seria representada — sobreviventes como uma lista de dicionários, inventário como uma lista de listas e o mapa de exploração como uma matriz (lista de listas de strings).
- 3. Divisão do problema em funções menores:** cada funcionalidade foi quebrada em subtarefas (por exemplo, "construir item" foi dividido em exibir receitas, verificar se há recursos suficientes, consumir os recursos e registrar o item construído), seguindo o princípio de que cada função deve ter uma única responsabilidade.
- 4. Implementação incremental:** o código foi escrito e testado funcionalidade por funcionalidade, começando pelo cadastro (mais simples) até chegar aos eventos aleatórios (que dependem das demais estruturas já existirem).
- 5. Testes manuais no terminal:** a cada função concluída, o menu principal era executado simulando o uso real do sobrevivente, testando entradas válidas e inválidas (nomes vazios, coordenadas fora do mapa, recursos insuficientes etc.).
- 6. Revisão final:** por último, o código foi revisado para eliminar repetições, padronizar mensagens exibidas ao usuário e garantir que nenhuma função dependesse de variáveis globais, apenas dos parâmetros recebidos.

3. Fontes consultadas

A lógica geral e a organização do código foram desenvolvidas a partir de conhecimento próprio de programação estruturada. Como apoio pontual, foi consultada a documentação oficial do Python (docs.python.org), especificamente as seções sobre estruturas de dados (listas e dicionários), o módulo random (para sorteios de recursos e eventos) e a formatação de strings com f-strings, para garantir o uso correto dessas ferramentas nativas da linguagem.

4. Explicação da lógica principal

O programa funciona em torno de um menu em loop que direciona a execução para funções específicas, cada uma responsável por uma única funcionalidade. As três estruturas de dados centrais são:

Lista de sobreviventes: cada sobrevivente é um registro (dicionário) dentro de uma lista, contendo nome, vida, fome e sede. Essa lista é aninhada, pois cada posição guarda um conjunto de informações relacionadas.

Mapa de exploração (matriz): uma grade 5x5 gerada aleatoriamente no início do jogo, em que cada célula contém um tipo de recurso ou está marcada como "Vazio". Ao explorar uma célula, o recurso ali contido é adicionado ao inventário e a célula é esvaziada, simulando o esgotamento daquele ponto do mapa.

Inventário (lista aninhada): representado como uma lista de pares [nome do recurso, quantidade]. Adicionar ou remover recursos consiste em percorrer essa lista procurando o nome correspondente e ajustando a quantidade.

A partir dessas estruturas, a construção de itens funciona comparando os requisitos de uma receita (também uma lista aninhada de recursos e quantidades) com o que existe no inventário; se todos os requisitos forem atendidos, os recursos são descontados e o item é adicionado à lista de itens construídos. Já os eventos aleatórios são organizados como um conjunto de funções independentes, sorteadas a cada rodada, que alteram o estado dos sobreviventes ou do inventário (por exemplo, reduzindo vida, restaurando sede ou adicionando recursos), simulando imprevistos do ambiente de sobrevivência. Por fim, a passagem do tempo reduz gradualmente a fome e a sede dos sobreviventes, criando uma pressão constante para que o jogador continue explorando e

gerenciando recursos.

5. Problemas encontrados e soluções utilizadas

Problema: risco de cadastrar o mesmo sobrevivente duas vezes com nomes escritos de forma diferente (ex.: "ana" e "Ana").

Solução: criação de uma função de normalização de nomes, que remove espaços extras e padroniza a capitalização antes de qualquer comparação ou cadastro.

Problema: ao tentar coletar um recurso, o usuário poderia digitar coordenadas fora dos limites do mapa ou valores não numéricos, causando erro de execução.

Solução: uso de tratamento de exceções na conversão da entrada para número inteiro, além de uma verificação explícita se a linha e a coluna informadas estão dentro dos limites da matriz antes de acessar a célula.

Problema: dificuldade inicial em decidir como representar o inventário, já que um dicionário seria mais simples de manipular, mas o enunciado exigia explicitamente o uso de listas aninhadas.

Solução: o inventário foi implementado como uma lista de listas, e funções auxiliares específicas (adicionar, remover, verificar quantidade) foram criadas para tornar essa estrutura tão fácil de usar quanto um dicionário seria.

Problema: risco de um sobrevivente tentar explorar o mapa mesmo estando com fome ou sede zeradas, o que não faria sentido dentro da lógica do jogo.

Solução: adicionada uma verificação no início da função de coleta de recursos que impede a exploração caso o sobrevivente esteja debilitado, informando o motivo ao usuário.

Problema: eventos aleatórios poderiam ser implementados como um bloco grande de if/elif, dificultando a manutenção e a leitura do código.

Solução: cada evento foi transformado em uma função independente, e um dicionário associa o nome do evento à função correspondente, permitindo sortear e executar o evento com poucas linhas de código.

6. Considerações finais

O sistema atende às cinco funcionalidades obrigatórias e utiliza corretamente listas aninhadas, matrizes e funções bem definidas, conforme exigido. A separação clara de responsabilidades entre as funções facilitou tanto os testes quanto a correção dos problemas encontrados durante o desenvolvimento, resultando em um programa executável diretamente pelo interpretador do Python, sem dependências externas.